Homework-9 (Optimization)

1. (a) Given, $\mathcal{E} = \left\{ x = \begin{pmatrix} x1 \\ x2 \end{pmatrix} : x^T Q x \leq 1 \right\}$, $Q = \begin{bmatrix} 1/9 & 0 \\ 0 & 1 \end{bmatrix}$

$$\Rightarrow \mathcal{E} = \begin{bmatrix} x1 & x2 \end{bmatrix} \begin{bmatrix} 1/9 & 0 \\ 0 & 1 \end{bmatrix} \left(\begin{bmatrix} x1 & x2 \end{bmatrix}^T \right) \leq 1$$

$$\Rightarrow \begin{bmatrix} \frac{x1}{9} & x2 \end{bmatrix} \begin{bmatrix} x1 \\ x2 \end{bmatrix} \leq 1$$

$$\Rightarrow \frac{x_1^2}{9} + x_2^2 \leq 1$$

$$\Rightarrow \boxed{x_1^2 + 9x_2^2 \leq 9}$$ ← "Please Check end for plot"

(b) Affine transform : $Ay = x$, $A = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$

$$\Rightarrow \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} y1 \\ y2 \end{bmatrix} = \begin{bmatrix} x1 \\ x2 \end{bmatrix}$$

$$\Rightarrow \begin{bmatrix} 3y1 \\ y2 \end{bmatrix} = \begin{bmatrix} x1 \\ x2 \end{bmatrix}$$ → Replacing in (a)

$$(3y1)^2 + 9y_2^2 \leq 9$$

$$\Rightarrow 9y_1^2 + 9y_2^2 \leq 9$$

$$\Rightarrow \boxed{y_1^2 + y_2^2 \leq 1}$$ ← "Please check end for plot"

(c) $\mathcal{E} = \left\{ x = \begin{pmatrix} x1 \\ x2 \end{pmatrix} : x^T Q x \leq 1 \right\}$, $Q = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$

$$\Rightarrow \mathcal{E} = \begin{bmatrix} x1 & x2 \end{bmatrix} \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix} \begin{bmatrix} x1 \\ x2 \end{bmatrix} \leq 1$$

$$\Rightarrow \begin{bmatrix} x1 + 0.9x2 & 0.9x1 + x2 \end{bmatrix} \begin{bmatrix} x1 \\ x2 \end{bmatrix} \leq 1$$

Qualcomm

$$\Rightarrow \boxed{x_1^2 + 1.8\, x_1 x_2 + x_2^2 \leq 1} \leftarrow$$ "Please check end for plot"

(d) Given, $\mathcal{E} : x^T Q x \leq 1$

We want of the form $y^T y \leq 1$.

Let $x = Ay$

Then, $x^T Q x \leq 1$

$$\Rightarrow (Ay)^T Q (Ay) \leq 1$$

$$\Rightarrow y^T A^T Q A y \leq 1$$

$$\Rightarrow y^T (A^T Q A) y \leq 1 \quad \text{—— (1)}$$

For (1) to be of the form $y^T y \leq 1$

$$\Rightarrow A^T Q A = I \quad (I \rightarrow \text{identity matrix})$$

Again, Q is p.s.d., so Q can be written as

$Q = v \lambda v^T$, where, $v \rightarrow$ eigen ~~value~~ vector matrix

$\lambda \rightarrow$ diagonal matrix with diagonals as eigenvalue

$$\therefore A^T (v \lambda v^T) A = I$$

Given, $Q = \begin{bmatrix} 1 & 0.9 \\ 0.9 & 1 \end{bmatrix}$. We use python to compute eigen value & eigen vector [attached at end]

Eigen values : $\lambda_1 = 1.9$, $\lambda_2 = 0.1$.

Eigen vector : $\begin{bmatrix} -1/\sqrt{2} & 1/\sqrt{2} \\ 1/\sqrt{2} & +1/\sqrt{2} \end{bmatrix}$

Qualcomm

Set $A = v\lambda^{-1/2}$, then $A^T(v\lambda v^T)A$

$$= \lambda^{-1/2} v^T v \lambda v^T v \lambda^{-1/2}$$

$$= I$$

$\therefore$ $A = v\lambda^{-1/2}$

$\hookrightarrow$ Putting this in python code we get

$$A = \begin{bmatrix} -2.236 & 0.513 \\ 2.236 & 0.513 \end{bmatrix}$$

2.

(a) We want to show G.D. on $f(x)$ is not equivalent to G.D. on $g(y) = f(Ay)$ even when starting points have relation $x_0 = Ay_0$

G.D. on $f(x)$: $x_{k+1} = x_k - \eta_k \nabla f(x_k)$ — (1)

G.D. on $g(y)$: $y_{k+1} = y_k - \eta_k \nabla g(y_k)$

same step size.

Now, $\nabla g(y_k) = A^T \nabla f(Ay)$

$$\therefore \quad y_{k+1} = y_k - \eta_k A^T \nabla f(Ay_k)$$

Multiply 'A' both sides, we get

$$Ay_{k+1} = Ay_k - \eta_k AA^T \nabla f(Ay_k)$$

$\because x_k = Ay_k$

$$\Rightarrow \quad Ay_{k+1} = x_k - \eta_k AA^T \nabla f(x_k) \quad \text{—— (2)}$$

Comparing (1) & (2) are equal if and only if

$$AA^T = I$$

$\hookrightarrow$ A must be orthogonal.

Qualcomm

<u>Example</u>: Let $f(x) = \frac{1}{2}(x_1^2 + 4x_2^2)$

and $A = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$, $x_0 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$, $\eta = 0.1$

$\because Ay_0 = x_0 \Rightarrow y_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

Now, GD on f:

$$\nabla f = \begin{bmatrix} x_1 \\ 4x_2 \end{bmatrix}\Bigg|_{\binom{2}{1}} = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$$

$$\therefore x_1 = x_0 - (0.1)\nabla f$$
$$= \begin{bmatrix} 2 \\ 1 \end{bmatrix} - (0.1)\begin{bmatrix} 2 \\ 4 \end{bmatrix} = \begin{bmatrix} 1.8 \\ 0.6 \end{bmatrix}$$

G.D. on g:

$$g(y) = f(Ay) = \frac{1}{2}\left[4y_1^2 + 4y_2^2\right]$$

$$\therefore \nabla g = \begin{bmatrix} 4y_1 \\ 4y_2 \end{bmatrix}\Bigg|_{\binom{1}{1}} = \begin{bmatrix} 4 \\ 4 \end{bmatrix}$$

$$\therefore y_1 = y_0 - (0.1)\nabla g$$
$$= \begin{bmatrix} 1 \\ 1 \end{bmatrix} - (0.1)\begin{bmatrix} 4 \\ 4 \end{bmatrix} = \begin{bmatrix} 0.6 \\ 0.6 \end{bmatrix}$$

But, $Ay = x \Rightarrow Ay_1 = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}\begin{bmatrix} 0.6 \\ 0.6 \end{bmatrix} = \begin{bmatrix} 1.2 \\ 0.6 \end{bmatrix} \neq x_1$

$\therefore x_1 \neq Ay_1$

Hence trajectories differ.

Qualcomm

(b) Now, for Newton method update is:

$$x_{k+1} = x_k - \left[\nabla^2 f(x_k)\right]^{-1} \nabla f(x_k) \quad \text{— on 'f'} \quad (1)$$

Given, $g(y) = f(Ay)$

$$\therefore \nabla g(y) = A^T \nabla f(Ay)$$

and $\nabla^2 g(y) = A^T \nabla^2 f(Ay) A$.

$\therefore$ Newton update on 'g':

$$y_{k+1} = y_k - \left[\nabla^2 g(y_k)\right]^{-1} \nabla g(y_k)$$

$$= y_k - \left[A^T \nabla^2 f(Ay_k) A\right]^{-1} A^T \nabla f(Ay_k)$$

Now, $\left(A^T \nabla^2 f(Ay_k) A\right)^{-1} = A^{-1}\left[\nabla^2 f(Ay_k)\right]^{-1} (A^T)^{-1}$

$$\therefore y_{k+1} = y_k - A^{-1}\left[\nabla^2 f(Ay_k)\right]^{-1} \underbrace{(A^T)^{-1} A^T}_{I} \nabla f(Ay_k)$$

$$\Rightarrow y_{k+1} = y_k - A^{-1}\left[\nabla^2 f(Ay_k)\right]^{-1} \nabla f(Ay_k)$$

Multiply by A on both sides,

$$Ay_{k+1} = Ay_k - \underbrace{AA^{-1}}_{I} \left[\nabla^2 f(Ay_k)\right]^{-1} \nabla f(Ay_k)$$

$$\Rightarrow Ay_{k+1} = Ay_k - \left[\nabla^2 f(Ay_k)\right]^{-1} \nabla f(Ay_k)$$

Also, $Ay_k = x_k$

$$\therefore Ay_{k+1} = x_k - \left[\nabla^2 f(x_k)\right]^{-1} \nabla f(x_k) \quad \text{— (2)}$$

Comparing (1) & (2) $Ay_{k+1} = x_{k+1} \ \forall \ k$

Hence Proved!

Qualcomm

3. Given, $\min_x \frac{1}{2} \|Ax - y\|_2^2 + \lambda \|x\|_1$

where, $x \in \mathbb{R}^d$, $y \in \mathbb{R}^n$

To show $\tilde{x}$ is optimal iff $\exists\, z \in \mathbb{R}^d$ s.t.

(a) $-A^T(y - A\tilde{x}) + \lambda z = 0$

(b) For every $i \in [d]$,

$$z_i = \begin{cases} \text{sign}(\tilde{x}_i), & \tilde{x}_i \neq 0 \\ |z_i| \leq 1, & \tilde{x}_i = 0 \end{cases}$$

The LASSO is written as $f(x) = g(x) + h(x)$

$$\nabla g(x) = \nabla\left(\frac{1}{2}\|Ax - y\|_2^2\right)$$

$$= A^T(Ax - y) \quad \text{——— (1)}$$

For $h(x) = \lambda \|x\|_1$, we compute the subgradient

Now, we know,

$$\partial |x_i| = \begin{cases} \{1\}, & x_i > 0 \\ \{-1\}, & x_i < 0 \\ [-1, 1], & x_i = 0 \end{cases}$$

$$\therefore \partial \|x\|_1 = \{ z \in \mathbb{R}^d : z_i \in \partial |x_i| \}$$

$$\therefore \partial h(x) = \lambda z, \text{ where } z_i = \begin{cases} \text{sign}(x_i), & x_i \neq 0 \\ |z_i| \leq 1, & x_i = 0 \end{cases}$$

——— (2)

For an optimal $\tilde{x}$,

$$0 \in \nabla g(\tilde{x}) + \partial h(\tilde{x}) \quad \left\{\text{Stationarity condition}\right\}$$

$\therefore$ there exists a z s.t.

$$0 = A^T(A\tilde{x} - y) + \lambda z$$

$$\Rightarrow -A^T(y - A\tilde{x}) + \lambda z = 0$$

where $z_i = \begin{cases} \text{sign}(\tilde{x}_i), & \tilde{x}_i \neq 0 \\ |z_i| \leq 1, & \tilde{x}_i = 0 \end{cases} \quad \forall i \in [d]$

<u>Hence Proved !!</u>

4. Given, LASSO problem,

$$\min_x \frac{1}{2}\|Ax - y\|_2^2 + \lambda \|x\|_1$$

(a). $f(x) = \underbrace{g(x)}_{\text{Smooth}} + \underbrace{h(x)}_{\text{non-smooth}}$

Subgradient method update is

$$x_{k+1} = x_k - \eta_k g_k$$

where, $g_k \in \partial f(x_k)$

From, previous problem

$$\nabla g = A^T(Ax - y)$$

& $\partial h = \lambda z$, where $z_i = \begin{cases} \text{sign}(x_i), & x_i \neq 0 \\ |z_i| \leq 1, & x_i = 0 \end{cases}$

Qualcomm

(subgradient)

$$\therefore \; g_k = A^T(Ax_k - y) + \lambda z_k$$

where, $z_k = \begin{cases} \text{sign}\left(x_k^{(i)}\right), & x_k^{(i)} \neq 0 \\ \in [-1, 1], & x_k^{(i)} = 0 \end{cases}$

$\therefore$ Update:

$$x_{k+1} = x_k - \eta_k\left(A^T(Ax_k - y) + \lambda z_k\right)$$

(b) For proximal gradient descent, update is

$$x^{t+1} = \text{prox}_{\eta h}\left(x^t - \eta \nabla g(x^t)\right)$$

For the gradient step on the smooth part,

$$x^t - \eta \nabla g(x^t) = x^t - \eta A^T(Ax^t - y)$$

let $v^t = x^t - \eta A^T(Ax^t - y)$ —— (1)

Now, $\text{prox}_{\alpha\|\cdot\|_1}(v)_i$ = ~~sign~~ soft thresholding operator

$$= \begin{cases} v_i - \alpha, & v_i > \alpha \\ 0, & -\alpha \leq v_i \leq \alpha \\ v_i + \alpha, & v_i < -\alpha \end{cases}$$

$\therefore$ Final update:

$$x_i^{t+1} = \begin{cases} v_i - \eta\lambda, & v_i > \eta\lambda \\ 0, & -\eta\lambda \leq v_i \leq \eta\lambda \\ v_i + \eta\lambda, & v_i < -\eta\lambda \end{cases}$$

where, $v^t = x^t - \eta A^T(Ax^t - y)$

Qualcomm

For convergence, a good choice of step size is

$$\eta \leq \frac{1}{L}, \text{ where } L \rightarrow \text{Lipschitz gradient}$$

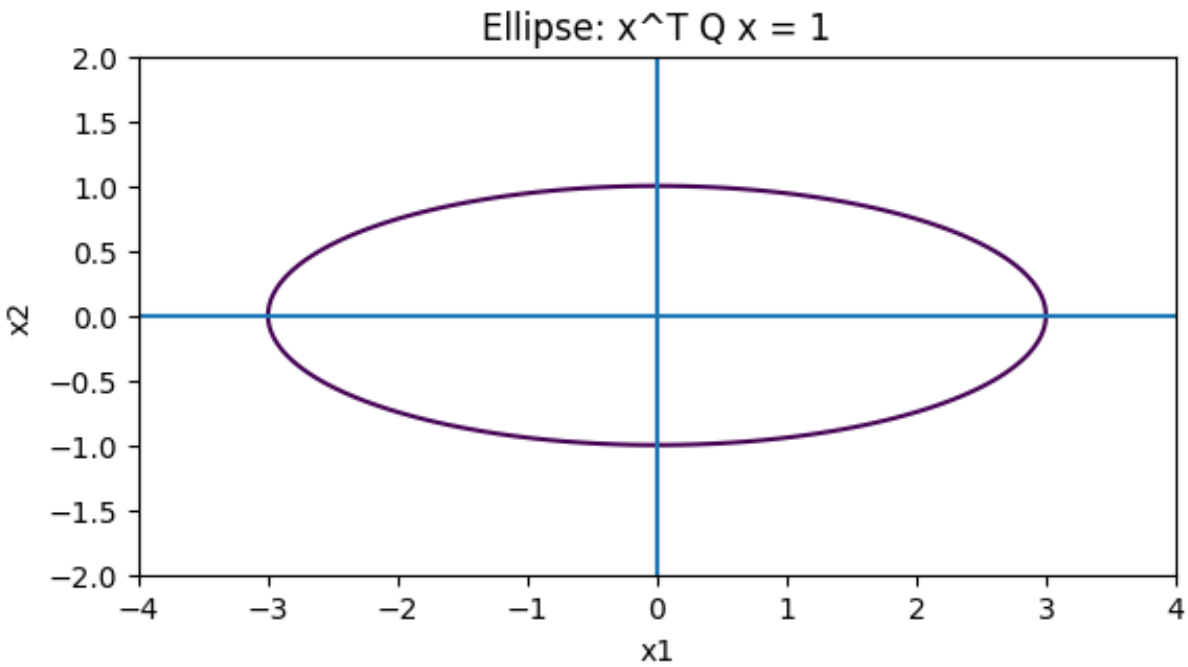
5.

The code and plot are attached.

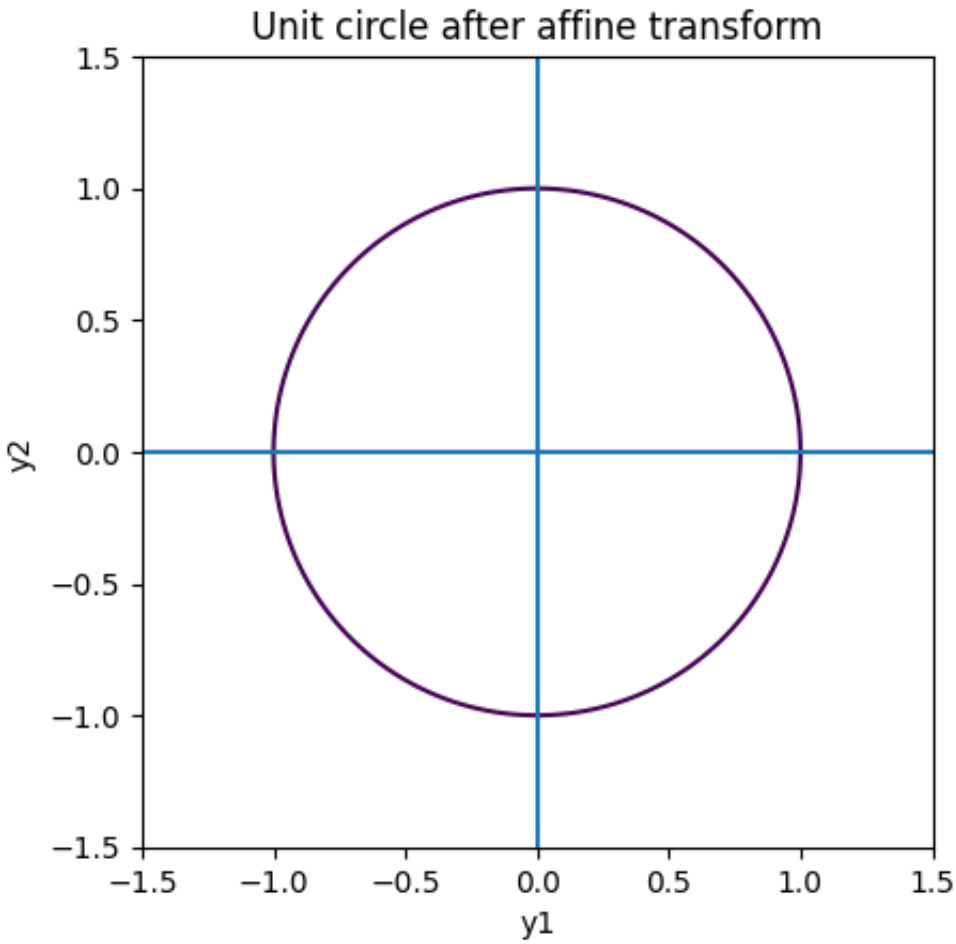
The $\|x^t - x^*\|_2$ vs iteration plot show that prox grad method converges faster.

Prox Grad is mostly linear to iteration $(1/t)$ whereas sub-gradient is $(1/\sqrt{t})$

```
import numpy as np
import matplotlib.pyplot as plt

Q = np.array([[1/9, 0],
              [0,   1]])

x1 = np.linspace(-4, 4, 400)
x2 = np.linspace(-2, 2, 400)

X1, X2 = np.meshgrid(x1, x2)

Z = (1/9)*X1**2 + X2**2

plt.figure()
plt.contour(X1, X2, Z, levels=[1])

plt.axhline(0)
plt.axvline(0)
plt.gca().set_aspect('equal')

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Ellipse: x^T Q x = 1")

plt.show()
```

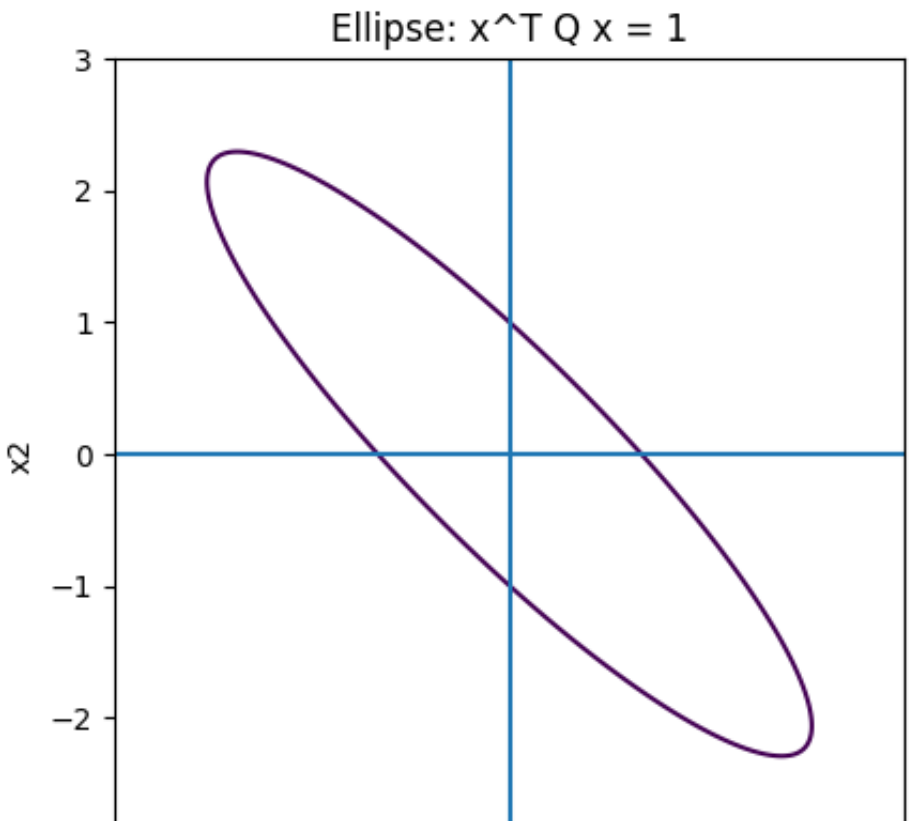


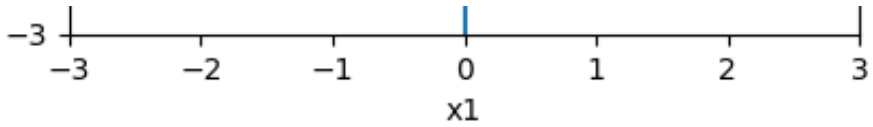
```
import numpy as np
import matplotlib.pyplot as plt

y1 = np.linspace(-1.5, 1.5, 400)
y2 = np.linspace(-1.5, 1.5, 400)

Y1, Y2 = np.meshgrid(y1, y2)

Z = Y1**2 + Y2**2

plt.figure()
plt.contour(Y1, Y2, Z, levels=[1])

plt.axhline(0)
plt.axvline(0)
plt.gca().set_aspect('equal')

plt.xlabel("y1")
plt.ylabel("y2")
plt.title("Unit circle after affine transform")

plt.show()
```




```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Define Q
Q = np.array([[1, 0.9],
              [0.9, 1]])

# Create grid
x1 = np.linspace(-3, 3, 400)
x2 = np.linspace(-3, 3, 400)

X1, X2 = np.meshgrid(x1, x2)

# Compute quadratic form x^T Q x
Z = (Q[0,0]*X1**2
     + 2*Q[0,1]*X1*X2
     + Q[1,1]*X2**2)

# Plot contour where x^T Q x = 1
plt.figure()
plt.contour(X1, X2, Z, levels=[1])

# formatting
plt.axhline(0)
plt.axvline(0)
plt.gca().set_aspect('equal')

plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Ellipse: x^T Q x = 1")

plt.show()
```

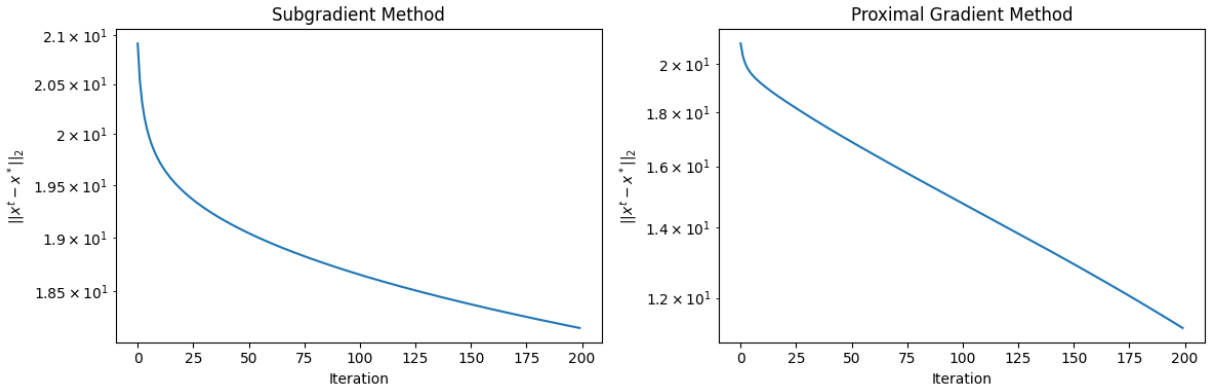





```
import numpy as np

Q = np.array([[1, 0.9],
              [0.9, 1]])

# eigen decomposition
eigvals, U = np.linalg.eigh(Q)
print(eigvals)
print(U)

# Lambda^{-1/2}
Lambda_inv_sqrt = np.diag(1/np.sqrt(eigvals))

# transformation matrix
A = U @ Lambda_inv_sqrt

print(A)
```

```
[0.1 1.9]
[[-0.70710678  0.70710678]
 [ 0.70710678  0.70710678]]
[[-2.23606798  0.51298918]
 [ 2.23606798  0.51298918]]
```

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

n = 50
d = 200

# true sparse vector
x_star = np.zeros(d)
nonzero_idx = np.random.choice(d, 5, replace=False)
x_star[nonzero_idx] = np.random.choice([-10, 10], 5)

# data generation
A = np.random.randn(n, d)
e = np.sqrt(0.1) * np.random.randn(n)
y = A @ x_star + e

lam = 10

# Lipschitz constant
L = np.linalg.norm(A.T @ A, 2)
```

```
T = 200

def soft_threshold(v, alpha):
    return np.sign(v) * np.maximum(np.abs(v) - alpha, 0)

############################
# Subgradient method (stable)
############################
x_sub = np.zeros(d)
sub_errors = []

for t in range(T):

    eta_subgrad = 1 / (L * np.sqrt(t+1))

    z = np.sign(x_sub)
    z[x_sub == 0] = 0

    grad = A.T @ (A @ x_sub - y) + lam * z

    x_sub = x_sub - eta_subgrad * grad

    sub_errors.append(np.linalg.norm(x_sub - x_star))

############################
# Proximal gradient method
############################
x_prox = np.zeros(d)
prox_errors = []

eta_prox = 1 / L

for t in range(T):

    v = x_prox - eta_prox * A.T @ (A @ x_prox - y)

    x_prox = soft_threshold(v, eta_prox * lam)

    prox_errors.append(np.linalg.norm(x_prox - x_star))

############################
# Side-by-side plots
############################
fig, axes = plt.subplots(1, 2, figsize=(12,4))

# Subgradient plot
axes[0].semilogy(sub_errors)
axes[0].set_title("Subgradient Method")
axes[0].set_xlabel("Iteration")
axes[0].set_ylabel(r"$||x^t - x^*||_2$")
```

```
# Proximal gradient plot
axes[1].semilogy(prox_errors)
axes[1].set_title("Proximal Gradient Method")
axes[1].set_xlabel("Iteration")
axes[1].set_ylabel(r"$||x^t - x^*||_2$")

plt.tight_layout()
plt.show()
```